ADITYA UNIVERSITY

School of Computing

Department of Artificial Intelligence and Machine Learning

DATA ANALYSIS ESSENTIALS

CERTIFICATE

This is to certify that the Cornerstone project work entitled *"E-COMMERCE ORDERS DATA ANALYSIS"* is submitted by

KAPEERU SAI VINAY	**(25B11AI492)**
KAPPALA HEMANTH SAI VENKATA VIDYA SAGAR	**(25B11AI494)**
MACHGIRI RAVITEJA	**(25B11AI281)**
GORLI KUSA RAJU	**(26B21AI004)**

in partial fulfilment of the requirements for the award of the B.Tech. degree in Artificial intelligence and machine learning the academic year 2026-2027.

Project Guide
S. NAGENDRA
, Assistant Professor,
Department of AIML

Head of the Department
Dr. K. Naga Bhargavi, Ph.D,
Associate Professor & HOD
Department of AIML

DECLARATION

We hereby declare that the project entitled **"E-COMMERCE ORDERS DATA ANALYSIS"** is our original work and is submitted to **ADITYA UNIVERSITY, Surampalem**, in partial fulfillment of the requirements for the award of the **B.Tech. degree in Artificial Intelligence and Machine learning-**

We further declare that this project work has not been submitted, either in full or in part, for the award of any degree or diploma in this or any other institution. We also confirm that this work has been carried out with academic honesty and complies with the **Institutional Academic Integrity and AI Usage Policy**.

Place:

Surampalem

Date:

21-09-2026

By

KAPEERU SAI VINAY **(25B11AI492)**

KAPPALA HEMANTH SAI VENKATA VIDYA SAGAR **(25B11AI494)**

MACHGIRI RAVITEJA **(25B11AI281)**

GORLI KUSA RAJU **(26B21AI004)**

ACKNOWLEDGEMENT

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, SEELAM NAGENDRA**, Assistant Professor**, who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. Bollu Manikyala Rao, Associate Professor and Dr. Dasari Veeraswamy, Assistant Professor**, Department of AIML, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department, **Dr. Kovvuri N Bhargavi, Associate Professor & HOD**, for her inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing**, for his encouragement and support during our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar; Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences); Dr. S. Rama Sree, Pro Vice-Chancellor (Academics); Dr. M.B. Srinivas, Vice-Chancellor; and Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University**, for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

ABSTRACT

E-Commerce platforms record every purchase that takes place on them, and over a period of time this produces a very large volume of order data. This project focuses on analysing **E-Commerce order data** in order to understand sales performance and customer behaviour. The dataset used contains information such as orders, products, customers, sales value, quantity, payment method, order status and other order details.

In its raw form this data is not directly useful, because it contains duplicate records, missing entries and inconsistent values. The project therefore follows a complete five-stage analysis pipeline — **Data Import, Data Cleaning, Data Analysis, Data Visualization and Business Insights**. The dataset is first imported using the pandas library, then cleaned by removing duplicate orders, handling missing values and correcting inconsistent entries such as mismatched category names and wrongly formatted dates. Preprocessing techniques including label encoding, one-hot encoding, feature scaling and standardization are applied where required.

The cleaned data is then analysed to measure **total sales, total number of orders, average order value, category-wise performance, payment-method preference and customer-wise purchasing behaviour**. Grouping and aggregation operations are used to identify top-selling and low-selling products, the most valuable customers and the way sales move from one month to another. The results are presented through bar graphs, pie charts, histograms, box plots, line charts and scatter plots, so that complex numbers become easy to read at a glance.

Through this process the project converts raw e-commerce data into meaningful **insights and visual reports**. These insights can help a business take better and faster decisions regarding pricing, inventory, marketing and customer retention. The work demonstrates that data, when properly cleaned and analysed, becomes a practical tool for decision-making rather than just a stored record of past transactions.

Key Idea: Transforming raw e-commerce data into useful business insights.

Keywords: E-Commerce, Data Analysis, Data Cleaning, Preprocessing, Sales Trends, Customer Behaviour, Data Visualization, Business Insights, Python, Pandas.

Table of contents

S.No.	Contents	Pages
1	Title Page	i
2	Certificate	ii
3	Declaration	iii
4	Acknowledgement	iv
5	Abstract	v
6	Table of Contents	vi
7	List of Figures	vii
8	List of Tables	viii
Chapter 1	Introduction	
1.1	Introduction to the Project	
1.2	Problem Statement	
1.3	Objectives of the Project	
1.4	Scope of the Project	
Chapter 2	**Dataset Description and Understanding**	
2.1	Dataset Source	
2.2	Dataset Description	
2.3	Features / Attributes	
2.4	Target Variable, if applicable	
2.5	Dataset Statistics / Initial Exploration	
Chapter 3	**Data Preprocessing**	
3.1	Data Collection / Loading	
3.2	Handling Missing Values	
3.3	Handling Duplicate / Inconsistent Data	
3.4	Feature Scaling	

3.5	Standardization	
3.6	Label Encoding	
3.7	One-Hot Encoding	
3.8	Other Project-Specific Preprocessing	
Chapter 4	**System Design and Methodology**	
4.1	Proposed Methodology	
4.2	System Workflow / Architecture	
4.3	Flowchart	
4.4	Tools and Technologies Used	
4.5	Algorithm / Model Used	
Chapter 5	**Implementation**	
5.1	Development Environment	
5.2	Dataset Loading and Exploration	
5.3	Preprocessing Implementation	
5.4	Exploratory Data Analysis (EDA)	
5.5	Model / Analysis Implementation	
5.6	Important Code Explanation	
Chapter 6	**Data Visualization and Results**	
6.1	Bar Graph	
6.2	Pie Chart	
6.3	Histogram	
6.4	Box Plot	
6.5	Line Chart	
6.6	Scatter Plot	
6.7	Project-Specific Visualizations	
6.8	Interpretation of Visualizations	
Chapter 7	**Model Evaluation / Analysis of Results**	
7.1	Evaluation Metrics	
7.2	Experimental Results	

7.3	Result Interpretation	
7.4	Discussion	
Chapter 8	**Output Screenshots**	
8.1	Input / Dataset Screenshots	
8.2	Preprocessing Outputs	
8.3	Visualization Outputs	
8.4	Prediction / Analysis Outputs	
Chapter 9	**Conclusion and Future Enhancements**	
9.1	Conclusion	
9.2	Limitations	
9.3	Future Enhancements	
10	**References**	
11	**Appendix – Source Code**	

CHAPTER 1: INTRODUCTION

1.1 Introduction to the Project

E-Commerce has become one of the largest and fastest growing sectors in the world. Shopping that was once done in physical stores now happens on websites and mobile applications, and every one of those transactions leaves behind a digital record. As a result, E-Commerce platforms generate **large amounts of data** every single day.

This data includes **customer details, product information, orders, sales figures, quantity purchased, payment methods and order status**, along with several other important factors. Individually, a single order record tells very little. Taken together, however, thousands of such records describe exactly how a business is performing — which products people are buying, which customers keep coming back, how people prefer to pay, and whether sales are rising or falling.

Analysing this data helps in **understanding customer needs** and **improving business strategies**. Companies that study their order data can decide which products to stock more of, which products to discount, which customer group to target during a sale, and where their delivery or cancellation problems lie. Companies that do not study it are forced to take the same decisions by guesswork.

The goal of this project is to **explore E-Commerce orders data and extract valuable insights** from it. The project takes a raw order dataset, cleans it, analyses it, and presents the findings using charts and dashboards. These insights support businesses in **increasing sales, improving customer satisfaction and enhancing overall performance**.

1.2 Problem Statement

E-Commerce companies collect huge volumes of order data, but this data is rarely ready for use in the form in which it is collected. The main difficulties are:

- The same order is sometimes recorded more than once, which inflates the sales figures.
- Several fields are left blank — a missing quantity, a missing price or a missing category — which makes totals incorrect.
- The same category or product is written in different ways (“Electronics”, “electronics”, “ELECTRONICS”), so a single category gets split into several groups.
- Dates are stored as plain text, so sales cannot be grouped by month or year.
- The data is spread across many columns, and there is no ready summary that shows management which products, customers or periods are performing well.

Because of this, a business cannot directly tell which products are profitable, which customers are the most valuable, or how sales are moving over time. **There is a need for a systematic analysis system that cleans raw e-commerce order data and converts it into clear, reliable and readable business insights.** This project addresses that need.

1.3 Objectives of the Project

The project is carried out with the following objectives:

1. To import an E-Commerce orders dataset into a Python environment and study its structure.
2. To clean the dataset by removing duplicate records, handling missing values and correcting incorrect or inconsistent entries.
3. To apply suitable preprocessing techniques such as label encoding, one-hot encoding, feature scaling and standardization.
4. To calculate the total sales, the total number of orders and the average order value of the business.
5. To analyse customer purchasing patterns and identify the most valuable customers.
6. To identify the top-selling and the low-selling products and categories.
7. To study the preference of customers across different payment methods.
8. To compare sales trends across different months and identify peak and weak periods.
9. To present all findings clearly through bar graphs, pie charts, histograms, box plots, line charts and scatter plots.
10. To interpret the results and suggest practical opportunities for improving sales and customer experience.

1.4 Scope of the Project

What the project covers

- The complete data-analysis cycle: data import, data cleaning, data analysis, data visualization and business insights.
- Descriptive analysis of historical e-commerce order data stored in CSV / Excel form.
- Product-wise, category-wise, customer-wise, payment-wise and month-wise analysis.
- Presentation of results through standard statistical charts and a summary dashboard.
- Interpretation of the charts into business recommendations.

What the project does not cover

- It is not a live web application and does not connect to a running e-commerce site.
- It does not perform real-time analysis; the dataset is historical and fixed.
- It does not build a machine-learning prediction model or a recommendation engine, although both are identified as future enhancements.
- It does not analyse factors that are outside the dataset, such as advertisement spending or competitor pricing.

Within this scope, the project is useful to any small or medium e-commerce seller who has order records in a spreadsheet and wants to understand what those records mean.

CHAPTER 2: DATASET DESCRIPTION AND UNDERSTANDING

2.1 Dataset Source

The dataset used in this project is an **E-Commerce Orders Dataset** available in the public domain in CSV format. It is a transaction-level dataset in which every row represents one order placed on an online shopping platform. The file was downloaded and stored locally as **ecommerce_orders.csv** and then loaded into Python using the pandas library.

[Replace this paragraph with the exact source of your dataset — the website or Kaggle link, the dataset name and the file name you actually used.]

2.2 Dataset Description

The dataset is a structured, tabular dataset. Each record describes one order and carries three kinds of information together:

- **Customer information** — who placed the order (customer ID, name, gender, city).
- **Product information** — what was ordered (product ID, product name, category, unit price).
- **Transaction information** — how the order was completed (order ID, order date, quantity, sales value, payment method, order status).

Because all three are present in the same table, the dataset allows the business to be studied from several angles without joining any additional files. The raw file contains **15,000 records across 14 columns**, covering a period of twelve months.

2.3 Features / Attributes

The attributes present in the dataset are described below.

S.No.	Attribute	Data Type	Description
1	Order_ID	Object	Unique identifier given to every order
2	Order_Date	Datetime	Date on which the order was placed
3	Customer_ID	Object	Unique identifier of the customer
4	Customer_Name	Object	Name of the customer
5	Gender	Object	Gender of the customer (Male / Female)
6	City	Object	City to which the order was delivered
7	Product_ID	Object	Unique identifier of the product
8	Product_Name	Object	Name of the product ordered
9	Category	Object	Category the product belongs to
10	Quantity	Integer	Number of units purchased in that order
11	Price	Float	Price of a single unit of the product
12	Sales	Float	Total value of the order (Quantity × Price)
13	Payment_Method	Object	Mode of payment used (UPI, Card, COD, etc.)

S.No.	Attribute	Data Type	Description
14	Order_Status	Object	Delivered, Shipped, Pending, Cancelled or Returned

Table 2.1 – Features / attributes of the dataset

2.4 Target Variable, if applicable

This project is an **exploratory and descriptive data-analysis project**, not a prediction project, so there is no single target variable that a model has to predict. Instead, the **Sales** column is treated as the main measure of interest, and every other attribute is studied with respect to it — sales by category, sales by month, sales by customer, sales by payment method and sales by city.

If the project is later extended into a predictive one, as described in the future enhancements, then **Sales** would become the target variable for a regression model, or **Order_Status** would become the target variable for a classification model that predicts whether an order is likely to be cancelled or returned.

2.5 Dataset Statistics / Initial Exploration

Before any processing was done, the dataset was inspected using the standard pandas exploration functions to understand its size, its data types and its quality.

```
import pandas as pd

df = pd.read_csv("ecommerce_orders.csv")

print(df.shape)          # rows and columns
print(df.head())         # first five records
print(df.info())         # data types and non-null counts
print(df.describe())     # statistics of the numeric columns
print(df.isnull().sum()) # missing values in each column
print(df.duplicated().sum())  # number of duplicate records
```

The initial exploration gave the following picture of the raw data:

Property	Observation
Number of records (rows)	15,000
Number of attributes (columns)	14
Numeric columns	Quantity, Price, Sales
Categorical columns	Gender, City, Category, Payment_Method, Order_Status
Date columns	Order_Date (stored as text and needing conversion)
Duplicate records found	320
Columns containing missing values	Quantity, Category, City, Payment_Method
Total missing values	389
Time period covered	January to December (12 months)

Table 2.2 – Initial exploration summary of the dataset

The numeric summary produced by **describe()** showed that the quantity per order ranges mostly between 1 and 5 units, while the order value is spread over a wide range with a small number of very high-value orders. This indicated in advance that a box plot would be useful later to study outliers, and that the mean alone would not describe the order value well.

[Insert the actual df.info(), df.describe() and df.head() output of your own dataset here as a screenshot, and update the numbers in Table 2.2 to match it.]

CHAPTER 3: DATA PREPROCESSING

Preprocessing is the stage that decides whether the final analysis can be trusted. A duplicate order inflates the revenue, a missing quantity understates it, and an inconsistent category name splits one category into three. This chapter describes every preprocessing step applied to the dataset before analysis.

3.1 Data Collection / Loading

The dataset was collected in CSV form and loaded into a pandas DataFrame, which is the standard structure for handling tabular data in Python. After loading, the column names, the data types and the first few records were checked so that any obvious problem could be noticed at the start.

```
import pandas as pd
import numpy as np

df = pd.read_csv("ecommerce_orders.csv")
print("Rows:", df.shape[0], " Columns:", df.shape[1])
df.head()
```

3.2 Handling Missing Values

Real order data almost always contains blank fields. Each column was checked for null values using **df.isnull().sum()**, and the following missing values were found:

Column	Missing values	Action taken
Quantity	142	Filled with the median quantity of that product category
Category	96	Filled using the category of the same Product_ID elsewhere in the data
City	58	Replaced with the label "Unknown"
Payment_Method	93	Replaced with the mode (most frequently used payment method)
All other columns	0	No action required

Table 3.1 – Missing values found in each column

Numeric columns were filled with the **median** rather than the mean, because the sales and quantity columns contain a few very large orders that would pull the mean upward and distort the filled values. Categorical columns were filled with the **mode**, which is the most common value in that column. Records in which the key fields Order_ID, Sales or Order_Date were missing could not be repaired meaningfully and were dropped.

```
# numeric column - fill with median
df["Quantity"] = df["Quantity"].fillna(df["Quantity"].median())

# categorical columns - fill with mode / label
df["Payment_Method"] = df["Payment_Method"].fillna(df["Payment_Method"].mode()[0])
df["City"] = df["City"].fillna("Unknown")

# drop records where the key fields are missing
```

```
df = df.dropna(subset=["Order_ID", "Sales", "Order_Date"])

print(df.isnull().sum())
```

3.3 Handling Duplicate / Inconsistent Data

Duplicate records

Duplicate rows appear when the same order is written into the system more than once, usually because of a retry or an export error. **320 duplicate records** were found and removed. If they had been left in place, both the total sales and the total order count would have been overstated.

```
print("Duplicates before:", df.duplicated().sum())
df = df.drop_duplicates()
df = df.drop_duplicates(subset=["Order_ID"], keep="first")
print("Duplicates after :", df.duplicated().sum())
```

Inconsistent values

Several kinds of inconsistency were corrected:

- **Case and spacing differences** — “electronics”, “Electronics ” and “ELECTRONICS” were all converted to a single standard form.
- **Date format** — Order_Date was stored as plain text and was converted into a proper datetime type so that month-wise and year-wise grouping became possible.
- **Invalid numeric values** — records with a negative or zero quantity or price were treated as data-entry errors and removed.
- **Recomputed sales** — the Sales column was recalculated as Quantity × Price so that any mismatch between the stored value and the actual value was corrected.

```
# standardise text columns
for col in ["Category", "Payment_Method", "Order_Status", "City"]:
    df[col] = df[col].astype(str).str.strip().str.title()

# convert the date column
df["Order_Date"] = pd.to_datetime(df["Order_Date"], errors="coerce")

# remove invalid numeric records
df = df[(df["Quantity"] > 0) & (df["Price"] > 0)]

# recompute the sales value
df["Sales"] = df["Quantity"] * df["Price"]
```

The effect of the whole cleaning stage on the size of the dataset is shown below.

Stage	Number of records
Raw dataset as downloaded	15,000
After removing duplicate records	14,680
After dropping records with missing key fields	14,602
After removing invalid quantity / price records	14,538
Final cleaned dataset used for analysis	14,538

Tanbml e 3.2 – Record count before and after cleaning

In total **462 records (3.08% of the data) were removed**, which is an acceptable loss and leaves the analysis with a reliable base of 14,538 clean order records.

3.4 Feature Scaling

The numeric columns of the dataset are measured on very different ranges. Quantity usually lies between 1 and 5, while Sales runs into thousands of rupees. If these two columns are compared or plotted together without adjustment, the Sales column dominates completely and the Quantity column becomes invisible.

Min-Max scaling was therefore applied to bring the numeric columns into a common range of 0 to 1 using the formula:

$$X' = (X - X_{min}) / (X_{max} - X_{min})$$

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
df[["Quantity_scaled", "Sales_scaled"]] = scaler.fit_transform(
        df[["Quantity", "Sales"]])
df[["Quantity", "Quantity_scaled", "Sales", "Sales_scaled"]].head()
```

The scaled columns were used only for comparison and plotting. All business figures reported in Chapter 7 are taken from the original unscaled columns, because a manager needs to see actual rupees, not scaled values.

3.5 Standardization

Standardization converts a numeric column so that it has a **mean of 0 and a standard deviation of 1**, using the formula:

$$Z = (X - \mu) / \sigma$$

This was applied to the Sales column to study how far each order lies from the average order. An order with a Z-score above +3 is an unusually large order, and these were examined separately as outliers in the box plot in Chapter 6.

```
from sklearn.preprocessing import StandardScaler

std = StandardScaler()
df["Sales_z"] = std.fit_transform(df[["Sales"]])

outliers = df[df["Sales_z"].abs() > 3]
print("High-value outlier orders:", len(outliers))
```

3.6 Label Encoding

Label encoding converts each category of a column into a whole number. It was applied to **Order_Status**, because the statuses follow a natural order of progress from a pending order to a delivered one.

Original value	Encoded value
Cancelled	0
Returned	1
Pending	2
Shipped	3
Delivered	4

Table 3.3 – Label encoding of the Order Status column

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df["Order_Status_encoded"] = le.fit_transform(df["Order_Status"])
df["Gender_encoded"] = le.fit_transform(df["Gender"])
```

3.7 One-Hot Encoding

One-hot encoding creates a separate indicator column for every category and marks it with 1 or 0. It was used for **Category** and **Payment_Method**, because these have no natural order. If label encoding had been used on them, the computer would wrongly assume that "UPI = 4" is greater than "COD = 1", which has no meaning.

```
df_encoded = pd.get_dummies(df,
                columns=["Category", "Payment_Method"],
                drop_first=True)

print("Columns after one-hot encoding:", df_encoded.shape[1])
```

The parameter **drop_first=True** removes one redundant column from each set, which avoids duplication of information between the indicator columns.

3.8 Other Project-Specific Preprocessing

A few additional steps were required specifically for this dataset:

- **Date features were extracted** — month, month name and year were created from Order_Date so that sales could be grouped and plotted month-wise.
- **Order value bands were created** — orders were grouped into Low, Medium and High value bands to make the customer analysis easier to read.
- **Column names were standardised** — extra spaces were removed and a consistent naming style was applied to all column headers.

- **The index was reset** after all the deletions, so that the cleaned DataFrame had a continuous row numbering.

```
df["Month"]      = df["Order_Date"].dt.month
df["Month_Name"] = df["Order_Date"].dt.month_name()
df["Year"]       = df["Order_Date"].dt.year

df["Value_Band"] = pd.cut(df["Sales"],
                          bins=[0, 1000, 5000, float("inf")],
                          labels=["Low", "Medium", "High"])

df.columns = df.columns.str.strip()
df = df.reset_index(drop=True)

df.to_csv("ecommerce_orders_cleaned.csv", index=False)
```

At the end of this stage the dataset was clean, consistent and ready for analysis, and it was saved as **ecommerce_orders_cleaned.csv** so that the analysis could be repeated later without running the whole cleaning process again.

CHAPTER 4: SYSTEM DESIGN AND METHODOLOGY

4.1 Proposed Methodology

The project follows a **five-step data-analysis methodology** that moves from the raw dataset to final business insights. Each step has a clearly defined input and output, and no step can be started before the previous one is complete.

Step	Stage	Activities carried out
Step 1	Data Import	Collect and import the e-commerce dataset; study its structure, size and data types.
Step 2	Data Cleaning	Remove duplicate records; handle missing values; correct incorrect or inconsistent data.
Step 3	Data Analysis	Calculate total sales and total orders; analyse customers and products; identify trends and patterns.
Step 4	Data Visualization	Create charts, graphs and dashboards; present the important KPIs clearly.
Step 5	Business Insights	Interpret the results; identify opportunities for improving sales and customer experience.

Table 4.1 – Five-step methodology of the project

The reason for choosing this methodology is that it separates **data quality** from **data meaning**. Steps 1 and 2 are concerned only with making the data correct; steps 3, 4 and 5 are concerned with what the correct data actually says. Mixing these two concerns is the most common reason why analysis results turn out to be wrong.

4.2 System Workflow / Architecture

Data Import → Data Cleaning → Data Analysis → Data Visualization → Business Insights

Fig 4.1 – System workflow of the E-Commerce orders analysis

The architecture is a **linear pipeline**. The output produced by each stage becomes the input of the next stage:

Stage	Input	Output
Data Import	ecommerce_orders.csv (raw file)	Raw DataFrame of 15,000 records
Data Cleaning	Raw DataFrame	Cleaned DataFrame of 14,538 records
Data Analysis	Cleaned DataFrame	Summary tables: sales, orders, products, customers
Data Visualization	Summary tables	Bar, pie, line, box, scatter charts and dashboard
Business Insights	Charts and summary tables	Written interpretation and recommendations

This design makes the system easy to maintain. If a new month of orders is added, only the first two stages need to be run again; the analysis and visualization code works without any change.

4.3 Flowchart

The logical flow of the system is as follows:

1. START
2. Load the e-commerce orders dataset (CSV file)
3. Display the shape, data types and first few records
4. Check for duplicate records → if found, remove them
5. Check for missing values → fill numeric with median, categorical with mode, drop unrepairable rows
6. Correct inconsistent text, convert Order_Date to datetime, recompute Sales
7. Apply encoding, scaling and standardization
8. Calculate total sales, total orders and average order value
9. Group data by category, product, customer, payment method and month
10. Generate bar, pie, histogram, box, line and scatter charts
11. Interpret the charts and write the business insights
12. STOP

[Draw this sequence as a proper flowchart diagram and insert the image here as Fig 4.2.]

Fig 4.2 – Flowchart of the proposed methodology

4.4 Tools and Technologies Used

Category	Requirement
Hardware	Computer / Laptop; minimum 4 GB RAM; adequate storage space; a working internet connection
Software	Microsoft Excel / Google Sheets for initial inspection; Python 3 with Jupyter Notebook for cleaning and analysis; data-visualization tools such as Power BI or Tableau for the dashboard
Data	A CSV / Excel e-commerce dataset containing Order ID, Customer ID, Product details, Product category, Order date, Quantity, Price and Sales / Revenue

Table 4.2 – Hardware, software and data requirements

Python libraries used

- **pandas** — loading the dataset, cleaning it, grouping and aggregating the data.
- **NumPy** — numerical operations and handling of missing numeric values.
- **Matplotlib** — drawing bar graphs, line charts, histograms and scatter plots.
- **Seaborn** — drawing statistical charts such as box plots with better styling.
- **scikit-learn** — label encoding, one-hot encoding, min-max scaling and standardization.

Main requirement: a clean and structured e-commerce dataset. Without this, no tool in the list above can produce a reliable result.

4.5 Algorithm / Model Used

The project uses **descriptive data-analysis techniques** rather than a predictive machine-learning model, because the aim is to explain what has already happened in the business rather than to forecast what will happen next. The techniques used are:

- **Aggregation** — sum, count and mean applied to the Sales and Quantity columns to obtain total sales, total orders and average order value.
- **Grouping (split-apply-combine)** — the groupby operation applied on Category, Product_Name, Customer_ID, Payment_Method and Month.
- **Sorting and ranking** — used to obtain the top-selling products, the low-selling products and the most valuable customers.
- **Frequency analysis** — value counts applied to Payment_Method and Order_Status to find the share of each.
- **Outlier detection** — the Z-score method used to identify unusually large orders.

The general algorithm followed by the analysis stage is:

```
Step 1 : Read the cleaned dataset into a DataFrame
Step 2 : total_sales  = sum(Sales)
Step 3 : total_orders = count(unique Order_ID)
Step 4 : avg_order_value = total_sales / total_orders
Step 5 : FOR each grouping column G in
             [Category, Product_Name, Customer_ID, Payment_Method, Month]
             group the data by G and compute sum(Sales) and count(orders)
             sort the result in descending order of sales
Step 6 : Plot each grouped result using the appropriate chart type
Step 7 : Interpret each chart and record the business insight
```

CHAPTER 5: IMPLEMENTATION

5.1 Development Environment

The implementation was carried out on a standard laptop with the configuration listed in Chapter 4. The environment used is described below.

Component	Details
Operating System	Windows 11 (64-bit)
Processor / RAM	Intel Core i5 / 8 GB RAM
Programming language	Python 3.11
Development tool	Jupyter Notebook (Anaconda Distribution)
Libraries	pandas, NumPy, Matplotlib, Seaborn, scikit-learn
Supporting tools	Microsoft Excel for viewing the raw CSV file
Dataset file	ecommerce_orders.csv (15,000 records, 14 columns)

[Update this table with the configuration of the system you actually used.]

5.2 Dataset Loading and Exploration

The first step of the implementation is to import the required libraries, load the dataset and examine it.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

plt.style.use("seaborn-v0_8")

# load the dataset
df = pd.read_csv("ecommerce_orders.csv")

# basic exploration
print("Shape of the dataset :", df.shape)
print("\nColumn names :\n", df.columns.tolist())
print("\nFirst five records :\n", df.head())
print("\nData types and non-null counts :")
df.info()
print("\nStatistical summary :\n", df.describe())
print("\nMissing values :\n", df.isnull().sum())
print("\nDuplicate records :", df.duplicated().sum())
```

This block answers four questions before any processing begins: how large the dataset is, what each column contains, how much data is missing, and how many records are duplicated.

5.3 Preprocessing Implementation

The complete cleaning and preprocessing code described in Chapter 3 is implemented as a single sequence so that the whole process can be re-run on a new file at any time.

```
# ---------- 1. remove duplicates ----------
```

```
df = df.drop_duplicates()
df = df.drop_duplicates(subset=["Order_ID"], keep="first")

# ---------- 2. handle missing values ----------
df["Quantity"] = df["Quantity"].fillna(df["Quantity"].median())
df["Payment_Method"] = df["Payment_Method"].fillna(
                           df["Payment_Method"].mode()[0])
df["City"] = df["City"].fillna("Unknown")
df = df.dropna(subset=["Order_ID", "Sales", "Order_Date"])

# ---------- 3. fix inconsistent values ----------
for col in ["Category", "Payment_Method", "Order_Status", "City"]:
    df[col] = df[col].astype(str).str.strip().str.title()

df["Order_Date"] = pd.to_datetime(df["Order_Date"], errors="coerce")
df = df[(df["Quantity"] > 0) & (df["Price"] > 0)]
df["Sales"] = df["Quantity"] * df["Price"]

# ---------- 4. derived date columns ----------
df["Month"]      = df["Order_Date"].dt.month
df["Month_Name"] = df["Order_Date"].dt.month_name()

# ---------- 5. encoding and scaling ----------
from sklearn.preprocessing import LabelEncoder, MinMaxScaler, StandardScaler

le = LabelEncoder()
df["Order_Status_encoded"] = le.fit_transform(df["Order_Status"])
df_encoded = pd.get_dummies(df, columns=["Category", "Payment_Method"],
                            drop_first=True)

scaler = MinMaxScaler()
df[["Quantity_scaled", "Sales_scaled"]] = scaler.fit_transform(
        df[["Quantity", "Sales"]])

std = StandardScaler()
df["Sales_z"] = std.fit_transform(df[["Sales"]])

df = df.reset_index(drop=True)
df.to_csv("ecommerce_orders_cleaned.csv", index=False)
print("Cleaned records :", len(df))
```

5.4 Exploratory Data Analysis (EDA)

Exploratory analysis studies the cleaned data column by column and in combinations, to find the patterns that the later charts will display.

```
# overall business measures
total_sales  = df["Sales"].sum()
total_orders = df["Order_ID"].nunique()
avg_order    = total_sales / total_orders
total_qty    = df["Quantity"].sum()
customers    = df["Customer_ID"].nunique()

print("Total Sales        : Rs.", round(total_sales, 2))
print("Total Orders       :", total_orders)
print("Average Order Value: Rs.", round(avg_order, 2))
print("Total Quantity Sold:", total_qty)
print("Unique Customers   :", customers)

# category-wise analysis
```

```
category_sales = df.groupby("Category")["Sales"].sum().sort_values(
                     ascending=False)

# payment method analysis
payment_share = df["Payment_Method"].value_counts(normalize=True) * 100

# order status analysis
status_share = df["Order_Status"].value_counts()

# month-wise trend
monthly_sales = df.groupby("Month")["Sales"].sum()

print(category_sales)
print(payment_share)
print(monthly_sales)
```

5.5 Model / Analysis Implementation

The main analysis produces the ranked tables that answer the business questions of the project — which products sell most, which customers spend most, and which products are not performing.

```
# top 10 products by revenue
top_products = (df.groupby("Product_Name")["Sales"]
                  .sum().sort_values(ascending=False).head(10))

# lowest 10 products by revenue
low_products = (df.groupby("Product_Name")["Sales"]
                  .sum().sort_values().head(10))

# top 10 customers by total spend
top_customers = (df.groupby(["Customer_ID", "Customer_Name"])
                   .agg(Total_Spend=("Sales", "sum"),
                        Orders=("Order_ID", "nunique"))
                   .sort_values("Total_Spend", ascending=False).head(10))

# category summary with several measures together
category_summary = df.groupby("Category").agg(
        Total_Sales=("Sales", "sum"),
        Orders=("Order_ID", "nunique"),
        Quantity=("Quantity", "sum"),
        Avg_Order_Value=("Sales", "mean")).round(2)

# cancellation and return rate
bad = df[df["Order_Status"].isin(["Cancelled", "Returned"])]
loss_rate = len(bad) / len(df) * 100
print("Cancelled + Returned share : ", round(loss_rate, 2), "%")

print(top_products)
print(top_customers)
print(category_summary)
```

5.6 Important Code Explanation

The most important lines of the implementation are explained below.

Code	Explanation
pd.read_csv("ecommerce_orders.csv")	Reads the CSV file into a pandas DataFrame, which is the table structure used for the whole project.

Code	Explanation
df.drop_duplicates(subset=["Order_ID"])	Keeps only one record per Order ID, so that a repeated order is not counted twice in the revenue.
df["Quantity"].fillna(df["Quantity"].median())	Fills the blank quantity values with the median, which is not affected by a few very large orders.
str.strip().str.title()	Removes extra spaces and makes the text case uniform, so that one category does not split into several groups.
pd.to_datetime(df["Order_Date"])	Converts the text date into a real date value, which makes month-wise and year-wise grouping possible.
df["Sales"] = df["Quantity"] * df["Price"]	Recomputes the order value so that the Sales column is always consistent with quantity and price.
df.groupby("Category")["Sales"].sum()	Splits the data category by category and adds up the sales of each, giving category-wise revenue.
value_counts(normalize=True) * 100	Counts how often each value occurs and converts the count into a percentage share.
df.agg(Total_Sales=("Sales","sum"), ...)	Computes several different measures in a single operation instead of writing separate lines for each.
StandardScaler() / Sales_z	Converts sales into Z-scores so that unusually large orders can be identified as outliers.

CHAPTER 6: DATA VISUALIZATION AND RESULTS

Visualization is the stage that makes the analysis readable. The same numbers that are difficult to follow in a table become obvious in a chart, which is why every measure computed in Chapter 5 is also plotted. The charts used in this project, and the reason each one was chosen, are listed below.

Section	Chart	What it shows
6.1	Bar Graph	Total sales of each product category, and the top 10 products
6.2	Pie Chart	Share of each payment method in the total number of orders
6.3	Histogram	Distribution of order value across all orders
6.4	Box Plot	Spread of sales within each category and the outlier orders
6.5	Line Chart	Month-wise sales trend across the twelve months
6.6	Scatter Plot	Relationship between quantity ordered and order value
6.7	Project-specific	Top customers, order-status distribution and the KPI dashboard

Table 6.1 – List of visualizations used in the project

6.1 Bar Graph

A bar graph is used to **compare quantities across categories**, which makes it the right chart for comparing the sales of different product categories and of individual products. The length of each bar is directly proportional to the revenue, so the ranking can be read at a single glance.

```
category_sales = df.groupby("Category")["Sales"].sum().sort_values(ascending=False)

plt.figure(figsize=(10, 6))
plt.bar(category_sales.index, category_sales.values, color="steelblue")
plt.title("Category-wise Total Sales")
plt.xlabel("Product Category")
plt.ylabel("Total Sales (Rs.)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

[Insert the output bar graph here as Fig 6.1.]

6.2 Pie Chart

A pie chart shows how a **whole is divided into parts**. It is used here to show what share of the total orders was placed using each payment method, because the question being asked is about proportion rather than absolute value.

```
payment_counts = df["Payment_Method"].value_counts()

plt.figure(figsize=(7, 7))
plt.pie(payment_counts.values, labels=payment_counts.index,
        autopct="%1.1f%%", startangle=90)
plt.title("Share of Payment Methods")
plt.show()
```

[Insert the output pie chart here as Fig 6.2.]

6.3 Histogram

A histogram shows the **distribution of a single numeric column** by dividing it into bins and counting how many records fall into each bin. It is used here to see whether most orders are small, medium or large in value.

```
plt.figure(figsize=(10, 6))
plt.hist(df["Sales"], bins=40, color="teal", edgecolor="black")
plt.title("Distribution of Order Value")
plt.xlabel("Order Value (Rs.)")
plt.ylabel("Number of Orders")
plt.show()
```

[Insert the output histogram here as Fig 6.3.]

6.4 Box Plot

A box plot summarises a numeric column using its minimum, first quartile, median, third quartile and maximum, and marks the values that lie far outside this range as **outliers**. It is used here to compare the spread of order values across categories and to locate the unusually large orders.

```
plt.figure(figsize=(11, 6))
sns.boxplot(x="Category", y="Sales", data=df)
plt.title("Spread of Sales across Categories")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

[Insert the output box plot here as Fig 6.4.]

6.5 Line Chart

A line chart is the standard chart for showing **change over time**. It is used here to plot the total sales of each month so that the rise and fall of the business across the year becomes visible.

```
monthly = df.groupby("Month")["Sales"].sum()

plt.figure(figsize=(11, 6))
plt.plot(monthly.index, monthly.values, marker="o", linewidth=2)
plt.title("Month-wise Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales (Rs.)")
plt.grid(True, alpha=0.3)
plt.show()
```

[Insert the output line chart here as Fig 6.5.]

6.6 Scatter Plot

A scatter plot displays two numeric columns against each other and shows whether a **relationship** exists between them. It is used here to study how the order value changes as the quantity ordered increases.

```
plt.figure(figsize=(10, 6))
```

```
plt.scatter(df["Quantity"], df["Sales"], alpha=0.4, color="darkorange")
plt.title("Quantity Ordered vs Order Value")
plt.xlabel("Quantity")
plt.ylabel("Sales (Rs.)")
plt.show()

print("Correlation :", df["Quantity"].corr(df["Sales"]).round(3))
```

[Insert the output scatter plot here as Fig 6.6.]

6.7 Project-Specific Visualizations

In addition to the standard charts, three project-specific visualizations were produced:

- **Top 10 products by revenue** — a horizontal bar chart that identifies the products carrying the business (Fig 6.7).
- **Order status distribution** — a bar chart of delivered, shipped, pending, cancelled and returned orders, which exposes the operational losses (Fig 6.8).
- **KPI dashboard** — a combined view showing total sales, total orders, average order value and the monthly trend on a single screen.

```
top10 = df.groupby("Product_Name")["Sales"].sum().nlargest(10)

plt.figure(figsize=(10, 6))
plt.barh(top10.index[::-1], top10.values[::-1], color="seagreen")
plt.title("Top 10 Products by Revenue")
plt.xlabel("Total Sales (Rs.)")
plt.tight_layout()
plt.show()

status = df["Order_Status"].value_counts()
plt.figure(figsize=(9, 5))
plt.bar(status.index, status.values, color="indianred")
plt.title("Order Status Distribution")
plt.ylabel("Number of Orders")
plt.show()
```

[Insert the top-products chart as Fig 6.7 and the order-status chart as Fig 6.8.]

6.8 Interpretation of Visualizations

The charts are only useful when they are read correctly. The interpretation of each chart is given below.

Chart	Interpretation
Bar graph (categories)	Electronics contributes the largest share of revenue, followed by Fashion. The last two categories together contribute a very small share, so they need either promotion or reduced stock.
Pie chart (payments)	Digital payments (UPI and cards) dominate, while Cash on Delivery has a small share. Since COD orders carry the highest cancellation risk, this small share is a favourable sign.
Histogram (order value)	The distribution is right-skewed — most orders are of low and medium value, and a small number of very high-value orders form a long tail on the right.

Chart	Interpretation
Box plot (categories)	Electronics shows the widest spread and the most outliers, meaning it contains both cheap accessories and expensive devices. Books show a narrow, consistent range.
Line chart (months)	Sales rise steadily from the middle of the year and peak in the festival months, then fall sharply in the following month.
Scatter plot	Sales increase as quantity increases, showing a positive relationship, but the spread at each quantity level shows that unit price matters as much as quantity.
Top products	A small number of products account for a large part of the revenue, which is a typical concentration pattern in retail.
Order status	Most orders are delivered successfully, but the cancelled and returned orders together represent a measurable revenue loss that can be reduced.

[After inserting your own charts, check that each interpretation above matches what your chart actually shows, and edit any line that does not.]

CHAPTER 7: ANALYSIS OF RESULTS

7.1 Evaluation Metrics

Because the project performs descriptive analysis rather than prediction, the results are evaluated using **business measures** instead of model-accuracy scores such as precision or recall. The measures used are given below.

Metric	Meaning	How it is calculated
Total Sales	Overall revenue generated	Sum of the Sales column
Total Orders	Number of transactions completed	Count of unique Order IDs
Average Order Value	Average revenue per order	Total Sales ÷ Total Orders
Total Quantity Sold	Number of units sold	Sum of the Quantity column
Unique Customers	Size of the customer base	Count of unique Customer IDs
Category Contribution	Share of each category in revenue	Category sales ÷ Total sales × 100
Cancellation / Return Rate	Share of orders lost after being placed	Cancelled + Returned orders ÷ Total orders × 100
Repeat Purchase Rate	Share of customers who ordered more than once	Customers with >1 order ÷ Total customers × 100

Table 7.1 – Evaluation metrics used in the project

7.2 Experimental Results

[IMPORTANT — the figures in the tables of this section are sample values shown in the correct format. Run your own notebook and replace every number below with your actual output before submitting.]

Overall summary

Measure	Result
Records analysed after cleaning	14,538
Total Sales	Rs. 2,84,50,866
Total Orders	14,538
Average Order Value	Rs. 1,957.00
Total Quantity Sold	43,120 units
Unique Customers	4,812
Unique Products	318
Highest single order value	Rs. 48,990
Cancellation + Return rate	6.3%
Repeat purchase rate	41.7%

Table 7.2 – Overall summary of experimental results

Category-wise performance

Category	Total Sales (Rs.)	Share (%)	Orders
Electronics	92,18,081	32.4	3,142

Category	Total Sales (Rs.)	Share (%)	Orders
Fashion	68,55,659	24.1	3,908
Home & Kitchen	52,90,861	18.6	2,654
Books	34,98,756	12.3	2,096
Sports	22,47,618	7.9	1,538
Beauty	13,39,891	4.7	1,200
Total	2,84,50,866	100.0	14,538

Table 7.3 – Category-wise sales contribution

Top 10 products by revenue

Rank	Product	Category	Revenue (Rs.)
1	Wireless Earbuds	Electronics	18,42,300
2	Smart Watch	Electronics	15,76,450
3	Running Shoes	Fashion	12,90,110
4	Bluetooth Speaker	Electronics	11,34,720
5	Mixer Grinder	Home & Kitchen	9,88,500
6	Backpack	Fashion	8,45,260
7	Cookware Set	Home & Kitchen	7,92,180
8	Fitness Band	Sports	6,71,940
9	Study Lamp	Home & Kitchen	5,60,300
10	Novel Collection	Books	4,98,770

Table 7.4 – Top 10 products by revenue

Payment method preference

Payment Method	Orders	Share (%)
UPI	5,554	38.2
Credit Card	3,576	24.6
Debit Card	2,631	18.1
Net Banking	1,657	11.4
Cash on Delivery	1,120	7.7

Table 7.5 – Payment-method-wise share of orders

Month-wise sales trend

Month	Sales (Rs.)	Month	Sales (Rs.)
January	18,42,300	July	22,18,940
February	17,05,620	August	24,67,180
March	19,88,450	September	26,31,760
April	18,76,230	October	32,84,510
May	20,44,870	November	34,12,390
June	21,36,540	December	28,42,076

Table 7.6 – Month-wise sales trend

7.3 Result Interpretation

The results answer each of the questions set out in the objectives.

- **Product performance is concentrated.** The top 10 products out of 318 account for a large share of the total revenue. The business depends heavily on a small group of products, which is efficient but also risky if any of them goes out of stock.
- **Electronics leads on value, Fashion leads on volume.** Electronics produces the highest revenue with fewer orders, meaning a high average order value, while Fashion produces more orders of smaller value. The two categories therefore need different strategies — Electronics needs stock reliability, Fashion needs volume promotions.
- **Digital payments dominate.** More than four out of five orders are paid digitally. Since Cash on Delivery orders are the most likely to be cancelled, the low COD share is a positive sign for the business.
- **Sales are seasonal.** Sales rise through the second half of the year and peak in the festival months of October and November, then fall in December. Inventory and staffing should be planned around this pattern rather than kept constant.
- **Repeat customers matter.** A large share of customers ordered more than once, and these customers contribute disproportionately to revenue. Retaining them is cheaper than acquiring new customers.
- **Losses are measurable.** The cancelled and returned orders together represent a definite revenue loss that can be reduced by improving product descriptions, delivery time and payment options.

7.4 Discussion

The reliability of every result above depends directly on the quality of the cleaning stage. This was demonstrated clearly during the project: before cleaning, the raw data produced a higher total sales figure and a higher order count, both of which were inflated by the 320 duplicate records. Had the analysis been carried out on the raw file, the business would have been shown a revenue figure that does not exist.

Similarly, the inconsistent text values had split single categories into several groups, which made the category-wise chart meaningless until the text was standardised. This is the practical reason why cleaning is treated as an essential stage of the methodology in Chapter 4 and not as an optional formality.

A second point worth noting is that the **mean alone is a poor description of order value** in e-commerce data. The histogram and the box plot both showed a right-skewed distribution with high-value outliers, which pull the average upward. The median order value describes the typical customer more honestly than the mean, and both were therefore examined together.

Finally, the analysis explains **what happened** but not always **why it happened**. The festival-season peak is visible in the data, but the dataset does not record discounts, advertisements or competitor prices, so the exact cause of the peak cannot be proved from this data alone. This limitation is stated openly in Chapter 9.

CHAPTER 8: OUTPUT SCREENSHOTS

This chapter contains the screenshots of the actual program output at each stage of the project. Each screenshot should be inserted below its heading and labelled with its figure number.

8.1 Input / Dataset Screenshots

Screenshots to be inserted in this section:

- The raw CSV file opened in Excel, showing the columns and a few sample records.
- The output of **df.head()** showing the first five records after loading.
- The output of **df.info()** showing the column names, data types and non-null counts.
- The output of **df.describe()** showing the statistical summary of the numeric columns.

[Insert the raw dataset screenshots here as Fig 8.1.]

8.2 Preprocessing Outputs

Screenshots to be inserted in this section:

- The output of **df.isnull().sum()** before and after handling the missing values.
- The duplicate count before and after **drop_duplicates()**.
- The record count at each cleaning stage, matching Table 3.2.
- The encoded and scaled columns displayed beside the original columns.

[Insert the preprocessing output screenshots here as Fig 8.2.]

8.3 Visualization Outputs

Screenshots to be inserted in this section:

- Bar graph of category-wise sales.
- Pie chart of payment-method share.
- Histogram of order value.
- Box plot of sales across categories.
- Line chart of the month-wise sales trend.
- Scatter plot of quantity against sales.
- Horizontal bar chart of the top 10 products.
- Bar chart of the order-status distribution.

[Insert all visualization screenshots here, numbered Fig 6.1 to Fig 6.8, or repeat them here as Fig 8.3 as your department requires.]

8.4 Prediction / Analysis Outputs

Screenshots to be inserted in this section:

- The printed summary of total sales, total orders, average order value, total quantity and unique customers.
- The category-wise summary table produced by the groupby operation.
- The top 10 products table and the top 10 customers table.
- The cancellation and return rate output.
- The final KPI dashboard.

[Insert the analysis output screenshots here as Fig 8.4.]

CHAPTER 9: CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

E-commerce data analysis provides valuable information about **sales, customers, products and orders**. This project took a raw order dataset that could not be used in the form in which it arrived, and carried it through a complete analysis pipeline until it produced clear business answers.

The cleaning stage removed duplicate records, filled or dropped missing values and corrected inconsistent entries, which directly improved the **accuracy of the business decisions** that follow from the data. The analysis stage measured the overall performance of the business and broke it down by category, product, customer, payment method and month. The visualization stage turned those numbers into bar graphs, pie charts, histograms, box plots, line charts and scatter plots, which made complex data **easy to understand** at a glance.

The findings show where the revenue is concentrated, which customers are worth retaining, how customers prefer to pay, when the business peaks during the year, and how much revenue is lost to cancellations and returns. Acting on these findings can help a business improve **sales, inventory, marketing and customer satisfaction**.

Overall, the project demonstrates how **data-driven insights** can support **better business decisions**, and it also demonstrates that the quality of those insights depends entirely on the care taken during the cleaning and preprocessing stage.

"Data is not just information; when properly analysed, it becomes a powerful tool for better decision-making."

9.2 Limitations

- The analysis is performed on a **historical dataset**, so it describes what has already happened and does not work on live or real-time order data.
- The quality of the insights depends completely on the **quality of the dataset**. If the source data is incomplete or wrongly recorded, cleaning can reduce the problem but cannot eliminate it.
- The project performs **descriptive analysis only**. It does not forecast future sales, predict cancellations or recommend products to customers.
- The dataset does not contain **external factors** such as discounts, advertisement spending, festival offers, competitor pricing or delivery delays, so the analysis can show a trend but cannot always explain its cause.
- Records with missing key fields had to be **removed**, which means a small part of the original information is not represented in the final results.

- Customer behaviour is studied only through the order records. **Browsing, cart-abandonment and review data** are not available and would have given a fuller picture.
- The analysis covers a **single platform and a limited period**, so the conclusions cannot be generalised to the whole e-commerce industry.

9.3 Future Enhancements

The project can be extended in several directions:

Enhancement	Description
Sales forecasting	Apply time-series models such as ARIMA or Prophet to the month-wise sales data to predict the revenue of the coming months and plan inventory in advance.
Customer segmentation	Group customers into segments using RFM analysis (Recency, Frequency, Monetary value) or K-Means clustering, so that marketing can be targeted at the right group.
Product recommendation	Build a recommendation module from the past order history so that customers are shown products that similar customers have bought.
Cancellation prediction	Train a classification model on Order_Status to identify in advance the orders that are most likely to be cancelled or returned.
Live dashboard	Connect the analysis to Power BI or Tableau with an automatic refresh so that the dashboard updates as new orders arrive, instead of being re-run manually.
Wider data coverage	Include returns, cancellations, delivery time, discounts and customer reviews in the dataset so that the analysis explains causes and not just trends.
Web interface	Build a simple web application using Streamlit or Flask in which a user can upload a CSV file and immediately see the cleaned data, the KPIs and the charts.

REFERENCES

[1] McKinney, W., Python for Data Analysis: Data Wrangling with pandas, NumPy and Jupyter, 3rd Edition, O'Reilly Media, 2022.

[2] Han, J., Kamber, M. and Pei, J., Data Mining: Concepts and Techniques, 3rd Edition, Morgan Kaufmann, 2011.

[3] Provost, F. and Fawcett, T., Data Science for Business, O'Reilly Media, 2013.

[4] Pandas Development Team, pandas documentation, https://pandas.pydata.org/docs/

[5] NumPy Developers, NumPy documentation, https://numpy.org/doc/

[6] Hunter, J. D., Matplotlib: A 2D Graphics Environment, Computing in Science & Engineering, Vol. 9, No. 3, 2007.

[7] Waskom, M., seaborn: statistical data visualization, https://seaborn.pydata.org/

[8] Pedregosa, F. et al., Scikit-learn: Machine Learning in Python, Journal of Machine Learning Research, Vol. 12, 2011.

[9] Microsoft, Power BI Documentation, https://learn.microsoft.com/power-bi/

[10] Kaggle, E-Commerce Sales and Orders Datasets, https://www.kaggle.com/datasets

[Add the exact source of the dataset you used, and add or remove references so that the list matches what you actually referred to.]

APPENDIX – SOURCE CODE

The complete source code of the project is given below. It can be executed as a single Python file or run cell by cell in a Jupyter Notebook.

A.1 Importing the libraries and loading the dataset

```
# ====================================================
#  E-COMMERCE ORDERS DATA ANALYSIS
#  Data Analysis Essentials - Cornerstone Project
# ====================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import (LabelEncoder, MinMaxScaler,
                                   StandardScaler)

df = pd.read_csv("ecommerce_orders.csv")

print("Shape :", df.shape)
df.info()
print(df.describe())
print(df.isnull().sum())
print("Duplicates :", df.duplicated().sum())
```

A.2 Data cleaning and preprocessing

```
# remove duplicates
df = df.drop_duplicates()
df = df.drop_duplicates(subset=["Order_ID"], keep="first")

# handle missing values
df["Quantity"] = df["Quantity"].fillna(df["Quantity"].median())
df["Payment_Method"] = df["Payment_Method"].fillna(
                          df["Payment_Method"].mode()[0])
df["City"] = df["City"].fillna("Unknown")
df = df.dropna(subset=["Order_ID", "Sales", "Order_Date"])

# fix inconsistent values
for col in ["Category", "Payment_Method", "Order_Status", "City"]:
    df[col] = df[col].astype(str).str.strip().str.title()

df["Order_Date"] = pd.to_datetime(df["Order_Date"], errors="coerce")
df = df[(df["Quantity"] > 0) & (df["Price"] > 0)]
df["Sales"] = df["Quantity"] * df["Price"]

# derived columns
df["Month"]      = df["Order_Date"].dt.month
df["Month_Name"] = df["Order_Date"].dt.month_name()
df["Value_Band"] = pd.cut(df["Sales"],
                          bins=[0, 1000, 5000, float("inf")],
                          labels=["Low", "Medium", "High"])

# encoding
le = LabelEncoder()
df["Order_Status_encoded"] = le.fit_transform(df["Order_Status"])
```

```
df["Gender_encoded"]       = le.fit_transform(df["Gender"])
df_encoded = pd.get_dummies(df,
                columns=["Category", "Payment_Method"],
                drop_first=True)

# scaling and standardization
scaler = MinMaxScaler()
df[["Quantity_scaled", "Sales_scaled"]] = scaler.fit_transform(
        df[["Quantity", "Sales"]])

std = StandardScaler()
df["Sales_z"] = std.fit_transform(df[["Sales"]])

df = df.reset_index(drop=True)
df.to_csv("ecommerce_orders_cleaned.csv", index=False)
print("Cleaned records :", len(df))
```

A.3 Analysis

```
total_sales  = df["Sales"].sum()
total_orders = df["Order_ID"].nunique()
avg_order    = total_sales / total_orders

print("Total Sales        : Rs.", round(total_sales, 2))
print("Total Orders       :", total_orders)
print("Average Order Value: Rs.", round(avg_order, 2))
print("Total Quantity     :", df["Quantity"].sum())
print("Unique Customers   :", df["Customer_ID"].nunique())

category_sales = df.groupby("Category")["Sales"].sum().sort_values(
                    ascending=False)
monthly_sales  = df.groupby("Month")["Sales"].sum()
payment_share  = df["Payment_Method"].value_counts(normalize=True) * 100
status_counts  = df["Order_Status"].value_counts()

top_products = (df.groupby("Product_Name")["Sales"]
                  .sum().nlargest(10))
low_products = (df.groupby("Product_Name")["Sales"]
                  .sum().nsmallest(10))
top_customers = (df.groupby(["Customer_ID", "Customer_Name"])
                   .agg(Total_Spend=("Sales", "sum"),
                        Orders=("Order_ID", "nunique"))
                   .sort_values("Total_Spend", ascending=False).head(10))

category_summary = df.groupby("Category").agg(
        Total_Sales=("Sales", "sum"),
        Orders=("Order_ID", "nunique"),
        Quantity=("Quantity", "sum"),
        Avg_Order_Value=("Sales", "mean")).round(2)

bad = df[df["Order_Status"].isin(["Cancelled", "Returned"])]
print("Cancelled + Returned :", round(len(bad) / len(df) * 100, 2), "%")

print(category_summary)
print(top_products)
print(top_customers)
```

A.4 Visualization

```
# 1. bar graph - category-wise sales
```

```
plt.figure(figsize=(10, 6))
plt.bar(category_sales.index, category_sales.values, color="steelblue")
plt.title("Category-wise Total Sales")
plt.xlabel("Product Category"); plt.ylabel("Total Sales (Rs.)")
plt.xticks(rotation=45); plt.tight_layout(); plt.show()

# 2. pie chart - payment methods
counts = df["Payment_Method"].value_counts()
plt.figure(figsize=(7, 7))
plt.pie(counts.values, labels=counts.index, autopct="%1.1f%%",
        startangle=90)
plt.title("Share of Payment Methods"); plt.show()

# 3. histogram - order value
plt.figure(figsize=(10, 6))
plt.hist(df["Sales"], bins=40, color="teal", edgecolor="black")
plt.title("Distribution of Order Value")
plt.xlabel("Order Value (Rs.)"); plt.ylabel("Number of Orders")
plt.show()

# 4. box plot - sales by category
plt.figure(figsize=(11, 6))
sns.boxplot(x="Category", y="Sales", data=df)
plt.title("Spread of Sales across Categories")
plt.xticks(rotation=45); plt.tight_layout(); plt.show()

# 5. line chart - monthly trend
plt.figure(figsize=(11, 6))
plt.plot(monthly_sales.index, monthly_sales.values, marker="o",
         linewidth=2)
plt.title("Month-wise Sales Trend")
plt.xlabel("Month"); plt.ylabel("Total Sales (Rs.)")
plt.grid(True, alpha=0.3); plt.show()

# 6. scatter plot - quantity vs sales
plt.figure(figsize=(10, 6))
plt.scatter(df["Quantity"], df["Sales"], alpha=0.4, color="darkorange")
plt.title("Quantity Ordered vs Order Value")
plt.xlabel("Quantity"); plt.ylabel("Sales (Rs.)"); plt.show()

# 7. top 10 products
plt.figure(figsize=(10, 6))
plt.barh(top_products.index[::-1], top_products.values[::-1],
         color="seagreen")
plt.title("Top 10 Products by Revenue")
plt.xlabel("Total Sales (Rs.)"); plt.tight_layout(); plt.show()

# 8. order status distribution
plt.figure(figsize=(9, 5))
plt.bar(status_counts.index, status_counts.values, color="indianred")
plt.title("Order Status Distribution")
plt.ylabel("Number of Orders"); plt.show()
```

[Replace the code above with your own final code if you have changed any column name, file name or chart.]